

Optimized tool selection
Reviewing the backend sources and configuration to infer the project purpose and next steps.

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend)

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/core/config.py)

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/pyproject.toml)

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/main.py)

Investigating backend package structure and API entrypoints to infer project scope and workflow.

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app)

Searched for files matching `\*\*/\*.py`, 11 matches

Gathering module contents and high-level structure to define the project idea clearly.

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/ai)

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/api)

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/chat)

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/parser)

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/graph)

Reviewing package entrypoints and available modules to infer app capabilities.

Read [](file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/CODELENS/backend/app/__init__.py)

Read [(file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/
CODELENS/backend/app/api/__init__.py)

Read [(file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/
CODELENS/backend/app/ai/__init__.py)

Read [(file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/
CODELENS/backend/app/chat/__init__.py)

Read [(file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/
CODELENS/backend/app/parser/__init__.py)

Checking repo metadata and source clues to reconstruct the product idea.

Searched for regex `CodeLens|codelens|OPENAI|github|neo4j|redis|FastAPI|
Graph|chat|parser|risk` ( `**/*.{py,md,txt}`), 34 results

Read [(file:///Users/srisaicharanp/Desktop/PENDING%20PROJECTS/
CODELENS/backend/requirements.txt)

Project Idea

This backend is a skeleton for `CodeLens AI`, an intelligent codebase understanding and analysis system.

What the project appears to be

- A FastAPI service exposing backend APIs for a code intelligence platform.
- Uses AI and code analysis to understand repositories, answer questions, and surface insights.
- Likely supports:
 - code parsing and analysis
 - repo ingestion via GitHub
 - graph-based relationships via Neo4j
 - caching or queueing with Redis
 - persistent storage in PostgreSQL
 - LLM-powered reasoning via OpenAI / Anthropic
 - a conversational/chat interface
 - risk detection or security / code quality scoring

Evidence from the code

- `app/main.py`
 - FastAPI app named `CodeLens AI`
 - `/health` and `/` endpoints only
- `app/core/config.py`
 - Config for PostgreSQL, Redis, Neo4j, OpenAI, GitHub OAuth, JWT, repo limits

- requirements.txt
 - `fastapi`, `uvicorn`, `openai`, `langchain`, `langgraph`
 - `neo4j`, `redis`, `asyncpg`, `celery`
 - `tree-sitter` for parsing code

Project scope implied by folder names

- `app/ai/` — AI prompt and model orchestration
- `app/api/` — API route definitions
- `app/chat/` — chat/conversational layer
- `app/parser/` — source code parsing and extraction
- `app/graph/` — code graph or dependency graph logic
- `app/risk/` — risk/vulnerability analysis

What is missing right now

The repo is mostly structural:

- most `app\*/\_\_init\_\_.py` files are empty
- no API routers are registered
- no parser, graph, AI, or risk logic exists yet
- no DB models, service layers, or actual route implementations

Suggested next step

1. Define the MVP

Choose the smallest useful product:

- "Upload a GitHub repo, parse it, build a code graph, and ask questions"
- or "Scan a repo for risks and return a summary report"

2. Implement core backend flow

Start with:

- `app/api/` router registration and endpoint definitions
- repo ingestion path
- parser that reads code files and extracts structure
- AI service that sends prompts to OpenAI
- graph builder that stores relationships in Neo4j

3. Validate infrastructure

Set up local/dev services:

- PostgreSQL
- Redis
- Neo4j
- OpenAI key / GitHub OAuth values
- run FastAPI app with `uvicorn app.main:app --reload`

4. Build one working feature

Example first feature:

- `POST /api/repos/import`
- clone GitHub repo
- parse code with `tree-sitter`
- persist metadata
- return a repo summary

Recommended concrete next move

Implement a minimal API route in `app/api` and wire it into `app/main.py` so the backend can do something measurable.

Example:

- `app/api/repo.py` with `/repos/import`
- `app/api/\_\_init\_\_.py` to include routers
- `app/main.py` to `include\_router(...)`

That gives you a real first step toward a usable product.

If you want, I can next help you draft the exact MVP feature list and scaffold the first API route.